
Air Quality Prediction System

Project Id: AIML-15

Name :Harsh Raunchela(Team Leader)

Class Roll Number: 47

University Roll Number :2028723

Section and Group: K – G2

Name : Devansh Dangwal

Class Roll Number:45

University Roll Number :2028700

Section and Group:K-G2

Name : Piyush Maurya

Class Roll Number:62

University Roll Number : 2028797

Section and Group: K-G1

TABLE OF CONTENTS

1. Problem Description	3
1.1 Dataset Description	4
1.2 Flow Diagram	4
2. Modules for Project Implementation	5
2.1 Modules Used	5
2.2 Platform Used	5
3. Machine Learning Models Used	6
3.1 Logistic Regression	6
3.2 K-Nearest Neighbors (KNN)	6
3.3 Support Vector Machine (SVM)	6
4. Evaluation Metrics Used	7
4.1 Accuracy	7
4.2 Precision	7
4.3 Recall	7
4.4 F1 Score	7
4.5 Confusion Matrix	7
5. Results and Visualizations	8
5.1 Model Accuracy Comparison	8
5.2 Correlation Heatmap	9
5.3 Confusion Matrix of Best Model (SVM)	9
5.4 ROC Curve Analysis	10
6. Conclusion and Future Scope	11
7. References	12
8. Appendix A: Best Model Parameters	13
Appendix B: Example Prediction Output	14

1. Problem Description

The Air Quality Prediction System is a Python-based machine learning application developed to automate the analysis and prediction of outdoor air quality levels based on pollutant concentration and meteorological data. Using core concepts of Machine Learning such as data preprocessing, feature selection, classification algorithms, model evaluation, and visualization, this project predicts the Air Quality Index (AQI) category for a given set of environmental readings.

The system classifies air quality into the following six categories as per Indian AQI standards:

Category	PM2.5 Range (µg/m³)	Health Implication
Good	0 – 30	Minimal impact
Satisfactory	31 – 60	Minor breathing discomfort
Moderate	61 – 90	Discomfort for sensitive people
Poor	91 – 120	Breathing discomfort for most
Very Poor	121 – 250	Respiratory illness on prolonged exposure
Severe	> 250	Serious health effects for all

1.1 Dataset Description

The dataset used in this project contains hourly air quality and weather observations collected over one year. The dataset consists of 8784 records and includes pollutant parameters such as PM2.5, NO₂, NO_x, NH₃, SO₂, CO, Ozone, Benzene, and meteorological features such as temperature, humidity, wind speed, pressure, and month.

After preprocessing and feature selection, 20 relevant features were selected for model training. The target variable is AQI_Label representing six AQI categories based on Indian CPCB standards.

1.2 Flow Diagram

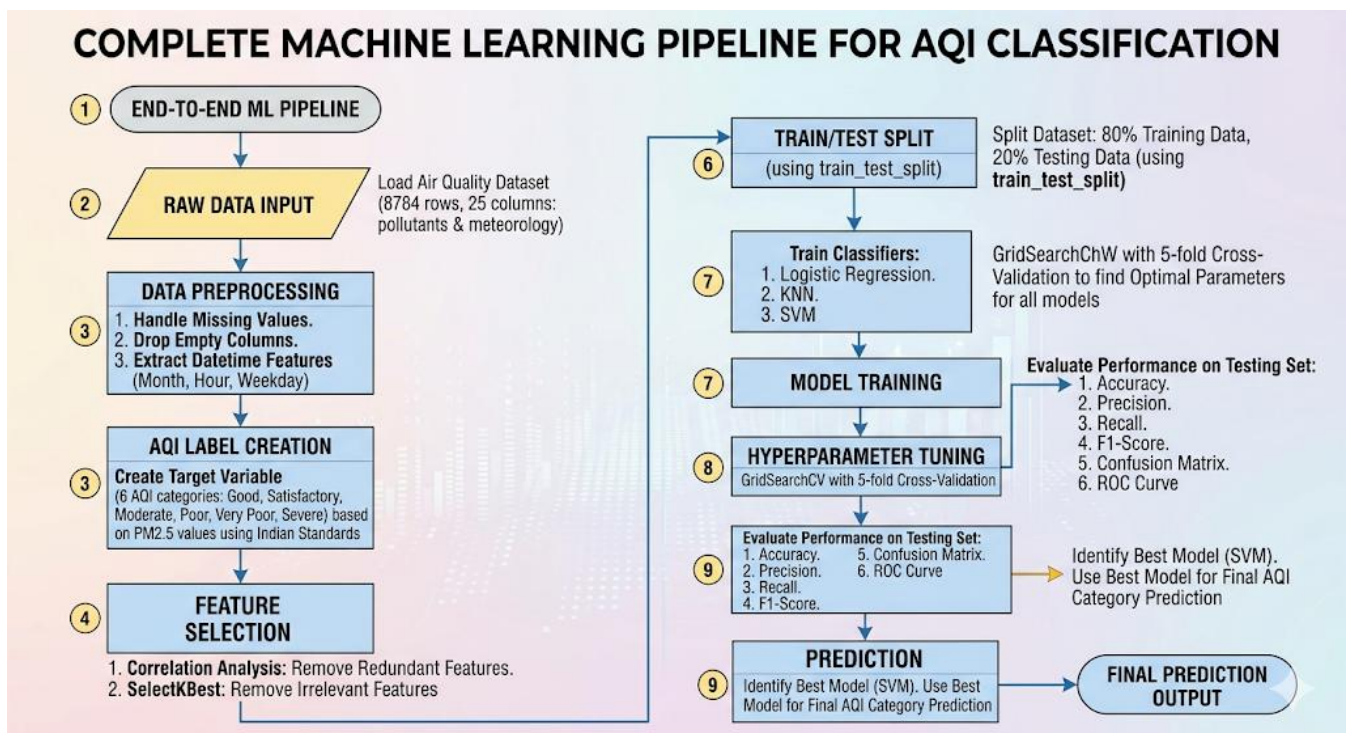


Fig.1 Sample flow diagram.

2. Modules for Project Implementation

2.1 Modules Used(mention all the inbuilt/ custom modules used and the purpose)

1. **Pandas** – Used for data loading, preprocessing, and manipulation.
Functions used: read_csv(), drop(), info(), describe(), head().
2. **NumPy** – Used for numerical operations and array processing.
3. **Matplotlib** – Used for plotting graphs such as bar charts and ROC curves.
4. **Seaborn** – Used for advanced visualizations like heatmaps and confusion matrices.
5. **Scikit-learn** – Main machine learning library used for:
 - LogisticRegression
 - KNeighborsClassifier
 - SVC
 - train_test_split
 - GridSearchCV
 - classification_report
 - confusion_matrix
 - roc_curve
 - SelectKBest
6. **Joblib** – Used for saving trained machine learning models for future prediction.

2.2 Platform Used: (mention all the platforms with the reason of using it)

- **Jupyter Notebook** – Used for coding, preprocessing, model training, and visualization due to its interactive environment.
- **GitHub** – Used for version control, code backup, and project management.
- **Python** – Used as the programming language for implementing the complete project.

3. Machine Learning Model/ Searching Technique Used(Describe each ML model/ searching technique used)

3.1 Logistic Regression

Logistic Regression is a classification algorithm that models the probability of each class using the sigmoid function. Despite its name, it is used for classification tasks. It works by finding the best decision boundary that separates classes.

- Type: Linear Classifier
- Best Parameters found: C = 10, solver = lbfgs
- Training Accuracy: 97.0% | Testing Accuracy: 97.0%

3.2 K-Nearest Neighbors (KNN)

KNN is a non-parametric algorithm that classifies a data point based on the majority class among its K nearest neighbors in the feature space. Distance metrics are used to find the nearest neighbors.

- Type: Instance-based Classifier
- Best Parameters found: K = 11, metric = manhattan, weights = uniform
- Training Accuracy: 83.1% | Testing Accuracy: 78.4%

3.3 Support Vector Machine (SVM)

SVM finds the optimal hyperplane that maximizes the margin between classes. It is one of the most powerful classifiers for high-dimensional data and works well with both linear and non-linear boundaries using kernels.

- Type: Margin-based Classifier
- Best Parameters found: C = 10, kernel = linear, gamma = scale
- Training Accuracy: 98.9% | Testing Accuracy: 98.6%
-

Model	Train Accuracy	Test Accuracy	F1 Score
Logistic Regression	97.0%	97.0%	0.97
KNN	83.1%	78.4%	0.77
SVM (Best)	98.9%	98.6%	0.98

•

4. Evaluation metrics used (Describe each evaluation metric used along with formulae)

- 1. Accuracy**
Measures the percentage of correctly classified samples.
 $\text{Accuracy} = (\text{TP} + \text{TN}) / \text{Total}$
- 2. Precision**
Measures correctness of positive predictions.
 $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
- 3. Recall**
Measures how many actual positives are correctly predicted.
 $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
- 4. F1** Score
Harmonic mean of Precision and Recall.
 $\text{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$
- 5. Confusion** Matrix
Displays correct and incorrect classifications for each AQI category.
- 6. ROC** Curve
Shows model discrimination capability using True Positive Rate vs False Positive Rate.

Metric	Formula	Description
Accuracy	$\text{TP} + \text{TN} / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$	Overall correct predictions out of all predictions
Precision	$\text{TP} / (\text{TP} + \text{FP})$	Of all predicted positives, how many are actually positive
Recall	$\text{TP} / (\text{TP} + \text{FN})$	Of all actual positives, how many were correctly predicted
F1 Score	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$	Harmonic mean of Precision and Recall
Confusion Matrix	N/A	Matrix showing correct and incorrect predictions per class
ROC Curve / AUC	Area under TPR vs FPR curve	Measures model's ability to distinguish between classes

5. Results and Visualizations

Model	Train Accuracy	Test Accuracy	F1 Score
Logistic Regression	97.0%	97.0%	0.97
KNN	83.1%	78.4%	0.77
SVM (Best)	98.9%	98.6%	0.98

5.1 Feature Correlation Heatmap

The heatmap below shows the correlation between all features. Dark red indicates high positive correlation (redundant features), dark blue indicates negative correlation.

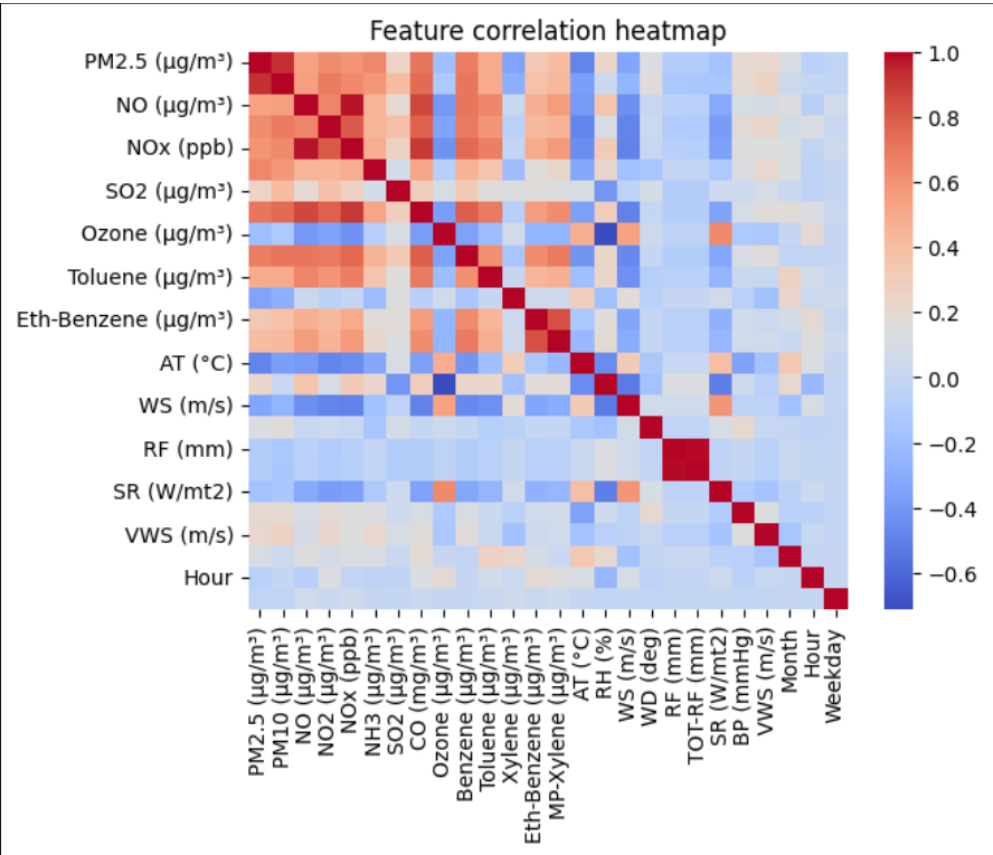


Fig. 1 — Feature Correlation Heatmap

5.2 Model Comparison Chart

The bar chart below compares the Accuracy and F1 Score of all three trained models.

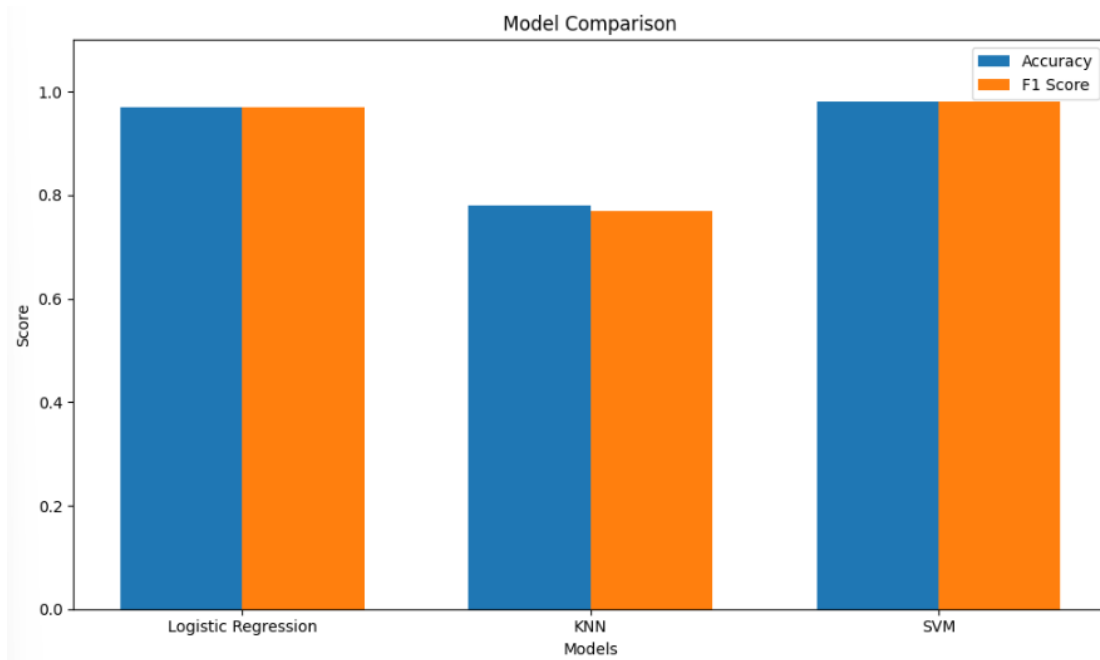


Fig. 2 — Model Comparison Chart (Accuracy & F1 Score)

5.3 Confusion Matrix (Best Model — SVM)

The confusion matrix shows predictions vs actual labels for the SVM model. Values on the diagonal represent correct predictions.

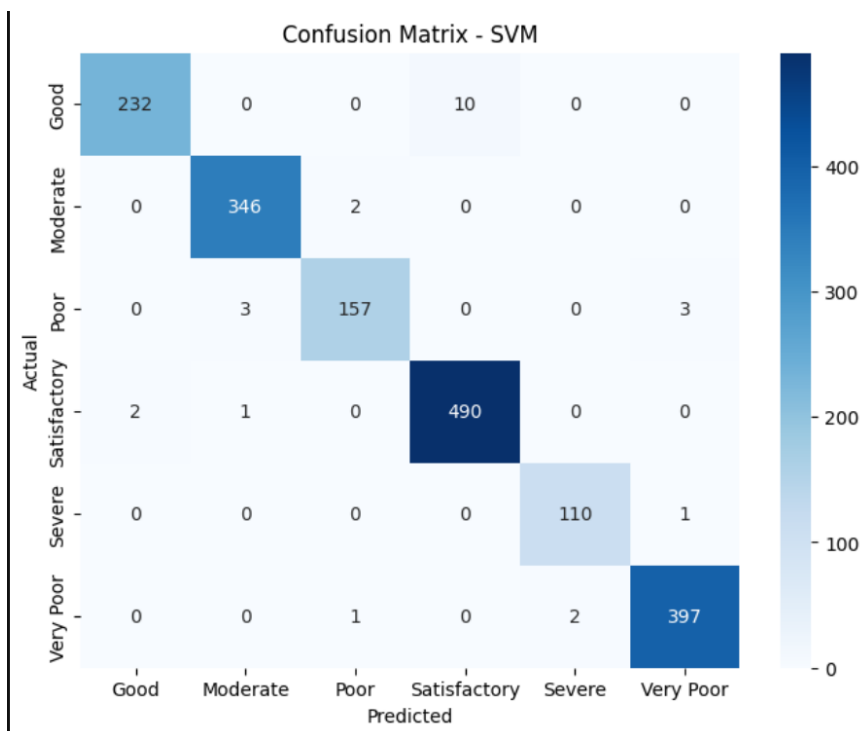


Fig. 3 — Confusion Matrix (SVM)

5.3 ROC CURVE (Best Model — SVM)

The ROC Curve is a graph that shows how well a classification model distinguishes between classes by plotting True Positive Rate against False Positive Rate at different thresholds.

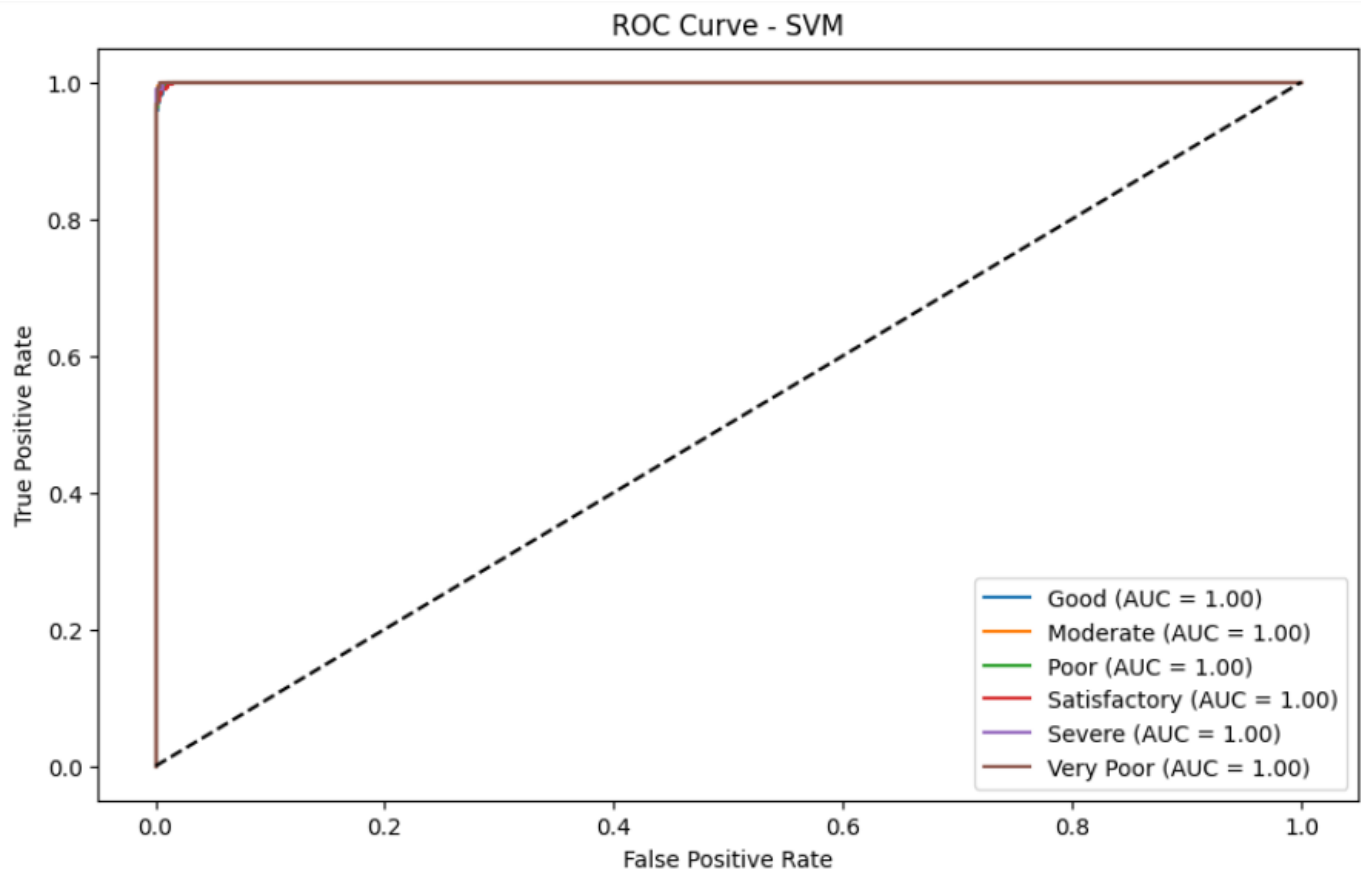


Fig. 4 — ROC Curve (SVM)

6. Conclusion and Future Scope:

This project successfully developed an end-to-end machine learning system for predicting air quality categories using pollutant and meteorological data. Data preprocessing, feature selection, model training, and evaluation were performed systematically.

Among all tested models, Support Vector Machine (SVM) achieved the highest accuracy of 98.6%, making it the best model for AQI prediction.

Future improvements may include:

- 1. Real-time IoT sensor integration**
- 2. Web or mobile application deployment**
- 3. Deep learning models for higher accuracy**
- 4. City-wise pollution forecasting**
- 5. Live AQI dashboard system**

7. References:

Books:

- Let Us Python – Yashavant Kanetkar
- Hands-On Machine Learning with Scikit-Learn – Aurelien Geron

Online Resources:

- Scikit-learn Documentation – <https://scikit-learn.org/>
- GeeksforGeeks – <https://www.geeksforgeeks.org/>
- Kaggle Air Quality Datasets – <https://www.kaggle.com/>
- Stack Overflow – <https://stackoverflow.com>
- TutorialsPoint – <https://www.tutorialspoint.com>

Appendix A: Model Settings and Deployment Notes

APPENDIX A: Model Settings and Deployment Notes

1. Best Model Selected:
The Support Vector Machine (SVM) classifier was selected as the final model because it achieved the highest accuracy of 98.6% among all tested models.
 2. Final Hyperparameters of SVM:
 - Kernel = Linear
 - C = 10
 - Probability = True
 - Random State = 42
 3. Other Models Evaluated:
 - Logistic Regression (Accuracy: 97.0%)
 - K-Nearest Neighbors (Accuracy: 78.0%)
 4. Data Preparation Settings:
 - Missing values handled during preprocessing
 - AQI labels created using Indian CPCB AQI standards
 - Feature scaling performed using StandardScaler
 - Feature selection applied using Correlation Analysis and SelectKBest
 - Final dataset contains 20 selected features
 5. Train-Test Split:
 - Training Data = 80%
 - Testing Data = 20%
 - Random State = 42
 6. Deployment Notes:
The trained models were saved using Joblib for future reuse without retraining.
- Saved Files:
- svm_model.pkl
 - lr_model.pkl
 - knn_model.pkl
 - scaler.pkl
 - label_encoder.pkl
7. User Prediction Script:
A Python script was developed to allow user interaction through terminal input. The script loads the saved SVM model and predicts AQI category based on user-provided values.
 8. Software Requirements:
 - Python 3.x
 - Jupyter Notebook
 - Required Libraries: pandas, numpy, matplotlib, seaborn, scikit-learn, joblib
 9. Future Deployment Scope:
The system can be deployed as:
 - Desktop Application
 - Web Application using Streamlit/Flask
 - Smart City AQI Dashboard
 - Real-time Sensor Based Monitoring System

Appendix B: Example User Interaction

The developed Air Quality Prediction System provides a simple command-line interface where the user can choose prediction options and receive AQI category output instantly.

Example Session:

AIR	QUALITY	PREDICTION	SYSTEM
Machine	Learning	Based	Project
		AQI	Classification

MAIN MENU

```
[1]          Predict          Using          Random          Dataset          Row
[2]          Predict          Using          Custom          Input
[3]          Show          Model          Performance
[4] Exit
```

User Selection: 2

CUSTOM AQI PREDICTION

Enter values between -2 to +3

PM2.5	Level	:	2.5
NO2	Level	:	2.0
CO	Level	:	1.8
Ozone	Level	:	1.0

Month : 1.0

Processing...

Predicted AQI Category: Very Poor

Another Example:

User Selection: 2

PM2.5	Level	:	-1.5
NO2	Level	:	-1.0
CO	Level	:	-1.0
Ozone	Level	:	0.0

Month : 0.0

Predicted AQI Category: Good

The system successfully predicts AQI categories based on varying pollution levels. Higher pollutant values generally lead to Poor, Very Poor, or Severe predictions, while lower values lead to Good or Satisfactory predictions.